Crear un videojuego

Nombre: _____ Fecha: _____

Qué vas a aprender

- Diseñar un videojuego sencillo definiendo objetivo, reglas, personajes y controles
- Programar el movimiento, las colisiones, la puntuación y las vidas
- Usar variables, clones y pantallas de inicio y fin
- Probar el juego con otras personas y mejorarlo

Para pensar

Los grandes videojuegos los hacen equipos de cientos de personas durante años. ¿Se puede hacer uno de verdad en una clase de 2º de ESO?

Explicación

Diseñar antes de programar

Antes de tocar un solo bloque, hay que **diseñar** el juego. Un buen diseño responde a estas preguntas: ¿cuál es el **objetivo** (atrapar, esquivar, llegar a la meta, sobrevivir un tiempo)?; ¿quién es el **protagonista** y quiénes son los enemigos u obstáculos?; ¿cuáles son las **reglas** (cómo se ganan puntos, cómo se pierden vidas, cuándo se gana y cuándo se pierde)?; ¿cuáles son los **controles** (flechas, ratón, espacio)?; y ¿cómo es el **escenario**? Todo esto se escribe en una ficha de diseño y se dibuja un **boceto** de la pantalla. Al conjunto de acciones que puede hacer el jugador y de reglas del juego se le llama *mecánica*. Consejo de los profesionales: empieza por una versión mínima que funcione (un personaje que se mueve y algo que atrapar) y añade lo demás después. Un juego pequeño terminado vale más que uno enorme a medias.

Elemento	Ejemplo: «Atrapa la fruta»
Objetivo	atrapar fruta que cae y conseguir 20 puntos
Protagonista	una cesta que se mueve abajo
Obstáculos	bombas que también caen
Reglas	fruta = +1 punto; bomba = −1 vida; 3 vidas
Controles	flechas izquierda y derecha
Fin	gana con 20 puntos; pierde con 0 vidas

undefined

Mover al protagonista

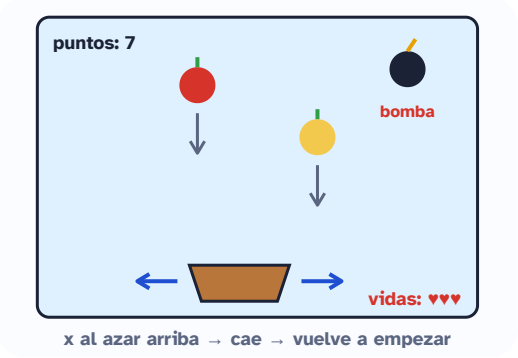
El primer paso es que el jugador pueda **controlar al personaje**. En un juego de atrapar, la cesta solo se mueve a izquierda y derecha en la parte de abajo: al hacer clic en la bandera, ir a x: 0 y: −150; por siempre { si ¿tecla flecha derecha presionada? entonces cambiar x en 10; si ¿tecla flecha izquierda presionada? entonces cambiar x en −10 }. Otra opción es que siga al ratón: por siempre { fijar x a (posición x del ratón) }. En un juego de laberinto o de nave, el personaje se mueve en las cuatro direcciones, y se usan también «cambiar y en». Si el personaje no debe salir del escenario, se puede limitar su posición con

condicionales («si posición x > 220 entonces fijar x a 220»). Para juegos de plataformas se añade la **gravedad**: una variable de velocidad vertical que va disminuyendo y hace que el personaje caiga si no hay suelo.

undefined

Objetos que caen y se mueven solos

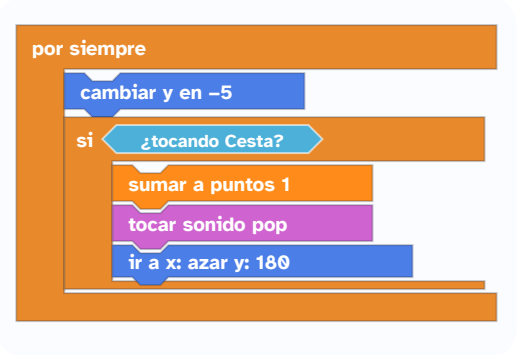
Los demás objetos del juego, como la fruta, las bombas o los enemigos, se mueven solos con sus propios programas. Una fruta que cae se programa así: al hacer clic en la bandera, por siempre { ir a x: (número al azar entre −220 y 220) y: 180; repetir hasta que (posición y < −170) { cambiar y en −5 } }. Así aparece arriba en un sitio al azar, baja hasta el suelo y vuelve a empezar. El bloque **«número al azar entre... y...»** (operadores) es clave en los videojuegos: hace que cada partida sea distinta y que el jugador no pueda adivinar lo que viene. Para que el juego se vuelva más difícil con el tiempo, se puede usar una variable **velocidad** y cambiar «cambiar y en −5» por «cambiar y en (0 − velocidad)», aumentando la velocidad cada vez que se consiguen 5 puntos. Los enemigos pueden moverse de lado a lado con «rebotar si toca un borde» o perseguir al jugador con «apuntar hacia Jugador».



Colisiones: cuando dos objetos se tocan

Una colisión ocurre cuando dos objetos del juego se tocan. Detectarlas es fundamental: es lo que hace que la cesta atrape la fruta, que el enemigo nos quite una vida o que lleguemos a la meta. En Scratch se detectan con los sensores **«¿tocando Objeto?»** y **«¿tocando el color...?»**, dentro de un condicional que está dentro de un bucle «por siempre». Por ejemplo, en el programa de la fruta: si ¿tocando Cesta? entonces { sumar a puntos 1; tocar sonido «pop»; ir a x: (al azar)

y: 180 }. Es importante que, después de una colisión, el objeto **se reinicie o desaparezca**; si no, seguiría tocando a la cesta y sumaría puntos muchas veces seguidas. El sensor de color es muy útil en laberintos: si las paredes son de un color, «si ¿tocando el color negro? entonces ir a la salida» devuelve al jugador al principio cada vez que choca.



Puntos, vidas y tiempo: las variables

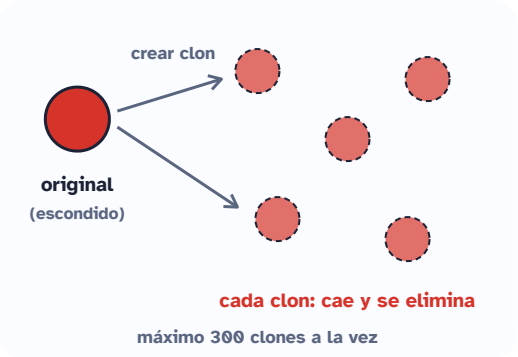
Las variables guardan los datos que cambian durante la partida. Las más habituales son **puntos**, **vidas**, **nivel** y **tiempo**. Se crean en la categoría Variables y se muestran en el escenario como marcadores. Hay que **inicializarlas** al empezar: al hacer clic en la bandera, dar a puntos el valor 0, dar a vidas el valor 3. Después se modifican con «sumar a puntos 1» o «sumar a vidas –1» cuando hay una colisión. Y se comprueban con condicionales para saber si se ha ganado o perdido: «si vidas = 0 entonces enviar "fin del juego"». Para un juego con tiempo límite se puede usar una variable **tiempo** que empieza en 30 y baja 1 cada segundo (repetir hasta que tiempo = 0 { esperar 1 segundos; sumar a tiempo –1 }). Una variable especial, **récord**, puede guardar la mejor puntuación: «si puntos > récord entonces dar a récord el valor puntos».

undefined

Clones: muchos objetos iguales

Si en el juego caen muchas frutas o aparecen muchos enemigos a la vez, no hace falta crear y programar decenas de objetos iguales. Scratch permite crear clones: copias de un objeto que aparecen durante la partida. Se usan tres bloques de Control: **«crear clon de mí mismo»**, que hace una copia; **«al comenzar como clon»**, un evento que indica qué debe hacer cada copia; y **«eliminar este clon»**,

que la borra cuando ya no hace falta. Por ejemplo, el objeto Fruta original se esconde y, por siempre, crea un clon cada segundo; cada clon va a una x al azar arriba, se muestra, cae, y si toca la cesta suma un punto y se elimina; si llega al suelo, también se elimina. Así puede haber muchas frutas a la vez en pantalla con un solo programa. Es importante eliminar los clones que ya no se usan, porque Scratch tiene un límite de 300 a la vez.



Pantallas de inicio, niveles y final

Un juego completo tiene varias **pantallas**: una de **inicio** con el título, las instrucciones y un botón «Jugar»; la del **juego**; y las de **victoria** («¡Has ganado!») y **derrota** («Game over»). En Scratch se hacen con **fondos** distintos del escenario y con **mensajes**: el botón «Jugar», al hacer clic, se esconde y envía «empezar»; todos los objetos, al recibir «empezar», se muestran y empiezan a funcionar; cuando las vidas llegan a 0, se envía «perder», el escenario cambia al fondo «Game over» y se detiene todo. Los **niveles** se hacen igual: cuando se consiguen 10 puntos, se cambia de fondo, se aumenta la velocidad de los enemigos o se añaden obstáculos. La **música y los efectos de sonido** (al atrapar, al perder una vida, al ganar) hacen el juego mucho más atractivo y ayudan al jugador a saber qué está pasando.

undefined

Probar, mejorar y compartir

Ningún juego sale bien a la primera. Los diseñadores de videojuegos trabajan por iteraciones: hacen una versión, la prueban, apuntan lo que falla o aburre, la mejoran y vuelven a probar. Al **probar** conviene preguntarse: ¿se entiende qué hay que hacer sin explicarlo?, ¿los controles responden bien?, ¿es demasiado fácil o

demasiado difícil?, ¿hay algún error (puntos que se suman dos veces, objetos que se quedan atascados)? Lo mejor es que lo pruebe **otra persona** mientras tú observas en silencio y anotas: verás cosas que no esperabas. Con sus comentarios se ajusta la **dificultad** (velocidad, número de vidas, tiempo) y se corrigen los fallos. Por último, se escriben las **instrucciones** y los **créditos** (quién lo ha hecho y de dónde salen las imágenes y sonidos) y se comparte. Un juego en el que otras personas se divierten es la mejor recompensa de un programador.

undefined

El ciclo de creación de un videojuego



Ejercicios

1. **¿Qué preguntas hay que responder al diseñar un videojuego?**

2. **¿Qué es la mecánica de un juego?**

3. **Escribe el programa para que un objeto siga al ratón horizontalmente.**

4. **¿Para qué sirve «número al azar» en un juego?**

5. **¿Qué es una colisión y con qué bloques se detecta?**

6. **¿Por qué hay que inicializar las variables al empezar?**

7. **¿Qué son los clones y qué tres bloques se usan?**

8. **¿Cómo se hacen las pantallas de inicio y fin?**

9. **Escribe cómo guardar el récord de puntos.**

10. **¿Qué es iterar en el diseño de videojuegos?**

Practica

1. **Lo primero para crear un juego es...**

   A) diseñarlo
   B) dibujar el fondo
   C) compartirlo

2. **Para que un objeto aparezca en sitios distintos se usa...**

   A) número al azar
   B) esperar
   C) esconder

3. **Una colisión se detecta con...**

   A) ¿tocando...?
   B) mover
   C) decir

4. **Los puntos se guardan en...**

   A) una variable
   B) un fondo
   C) un disfraz

5. **Al empezar, las vidas deben...**

   A) inicializarse
   B) borrarse
   C) esconderse

6. **Para muchas frutas iguales se usan...**

   A) clones
   B) fondos
   C) mensajes

7. **Cada clon empieza con el evento...**

   A) al comenzar como clon
   B) al hacer clic en ⚑
   C) al recibir

8. **El cambio de pantalla de inicio a juego se hace con...**

   A) mensajes y fondos

B) sonidos

C) el lápiz

9. **Para aumentar la dificultad se puede...**

A) subir la velocidad

B) quitar los controles

C) borrar el objetivo

10. **Probar, mejorar y volver a probar es...**

A) iterar

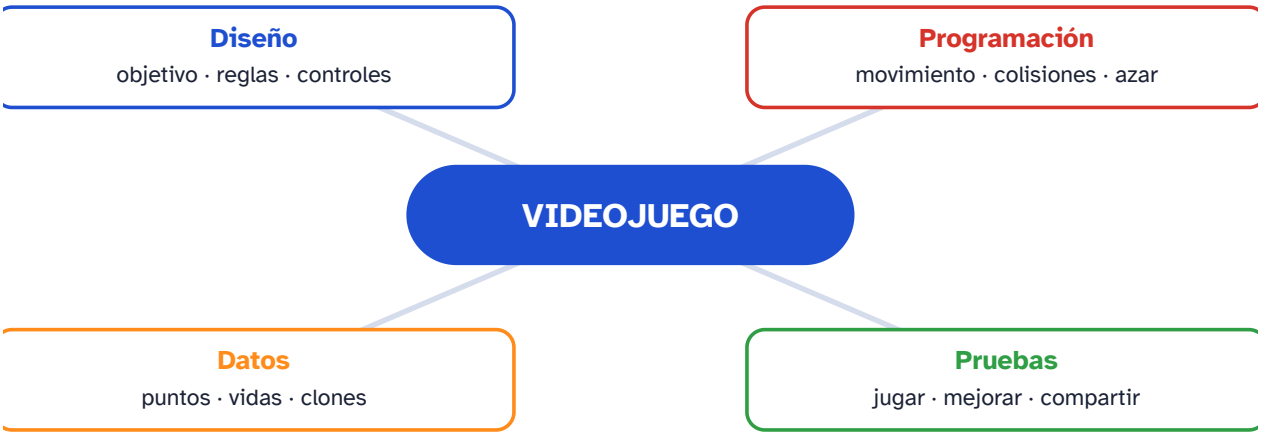
B) compilar

C) clonar

Explícalo con tus palabras

Incluye: Qué hay que decidir al diseñar un videojuego; Cómo se detecta una colisión y qué hay que hacer después; Para qué sirven las variables y los clones en un juego.

Resumen



Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

Mecánica

Acciones del jugador y reglas del juego.

Colisión

Cuando dos objetos se tocan.

Número al azar

Hace que cada partida sea distinta.

Variable puntos

Guarda la puntuación; se inicializa a 0.

Vidas

Variable que baja al chocar; a 0, fin.

Clon

Copia de un objeto creada durante el juego.

Pantallas

Inicio, juego y fin con fondos y mensajes.

Iterar

Probar, mejorar y volver a probar.

Soluciones

1. **A)** Diseño antes de programar.
2. **A)** Azar.
3. **A)** Sensores.
4. **A)** Variable.
5. **A)** Por ejemplo, a 3.
6. **A)** Clones.
7. **A)** Al comenzar como clon.
8. **A)** Mensajes y fondos.
9. **A)** Más velocidad.
10. **A)** Iterar.

Soluciones de los ejercicios

1. Objetivo, protagonista y obstáculos, reglas, controles, escenario y cómo se gana o se pierde.
2. El conjunto de acciones del jugador y reglas del juego.
3. Bandera; por siempre { fijar x a posición x del ratón }.
4. Para que cada partida sea distinta e imprevisible.
5. Cuando dos objetos se tocan; con «¿tocando Objeto?» o «¿tocando el color?».
6. Para que no conserven los valores de la partida anterior.
7. Copias de un objeto: crear clon, al comenzar como clon, eliminar este clon.
8. Con fondos distintos y mensajes que avisan a los objetos.
9. Si puntos > récord entonces dar a récord el valor puntos.
10. Repetir el ciclo de probar y mejorar varias veces.